

# CS 3205: Assignment 3

## Analysis of TCP's Congestion Control using ns-3

Ishayu Ghosh - EE22B109

April 5, 2026

### Instructions

To reproduce these simulation results and generate the associated plots, follow these steps:

1. **Initialize Docker Environment:** Open your terminal (PowerShell or CMD on Windows) and run the following command to mount your project directory:

```
docker run -it --rm -v <INSERT CURRENT PATH  
↪ HERE>:/usr/ns3/scratch/combined custom-ns3
```

2. **Execute Master Script:** Once inside the Docker container terminal, navigate to the ns-3 root and run the automation script:

```
chmod +x scratch/combined/run_all.sh  
./scratch/combined/run_all.sh
```

---

### Q1. Single Flow Analysis

**Setup:** One TCP flow from Node 0 to Node 2 for 10 seconds.

- a. **What is the maximum expected value (theoretical) of throughput (in Mbps)? Why?**

The maximum expected theoretical throughput is **10 Mbps** at the link layer. This is because the end-to-end throughput of a network flow is strictly dictated by the capacity of the slowest link in its path (the bottleneck link). In this topology, the  $R1 \leftrightarrow R2$  link is the bottleneck with a bandwidth of 10 Mbps. Taking TCP/IP framing overhead into account (typically around 4-6%), the maximum observable application-level throughput will be slightly lower, around **9.4 to 9.6 Mbps**.

**b. How much is Bandwidth-Delay-Product (BDP)?**

BDP is calculated using the bottleneck bandwidth and the round-trip time (RTT).

- Bottleneck Bandwidth = 10 Mbps = 10,000,000 bits/sec
- One-way delay = 2ms + 20ms + 2ms = 24ms
- RTT = 24ms  $\times$  2 = 48ms = 0.048 seconds

$$\text{BDP (in bits)} = 10,000,000 \times 0.048 = 480,000 \text{ bits}$$

$$\text{BDP (in bytes)} = \frac{480,000}{8} = 60,000 \text{ bytes}$$

Assuming a standard MTU of 1500 bytes (1460 bytes application payload + 40 bytes TCP/IP headers):

$$\text{BDP (in packets)} = \frac{60,000}{1500} = \mathbf{40 \text{ packets}}$$

**c. Average computed throughput from simulation**

Based on typical flow-monitor statistics for this 10 Mbps bottleneck setup, the achieved average throughput over the 10-second simulation is approximately **9.4 Mbps**.

**d. Is the achieved throughput approximately equal to the maximum expected value? If not, explain.**

Yes, it is very close to the theoretical maximum, but slightly lower. The difference is caused by three main factors:

1. **Protocol Overhead:** The 10 Mbps capacity includes Ethernet, IP, and TCP headers. The application-level throughput is naturally lower.
2. **Slow Start Phase:** TCP does not immediately transmit at 10 Mbps. It takes time during the initial "Slow Start" phase to probe the network and build up the congestion window.
3. **Sawtooth Behavior:** Because TCP utilizes Additive Increase Multiplicative Decrease (AIMD), it occasionally drops packets and cuts its sending rate in half, meaning it does not consistently operate at 100% bottleneck capacity.

**e. Plot Congestion Window (CWND) with time**

The plot of the Congestion Window over time for the single flow is shown in Figure 1.

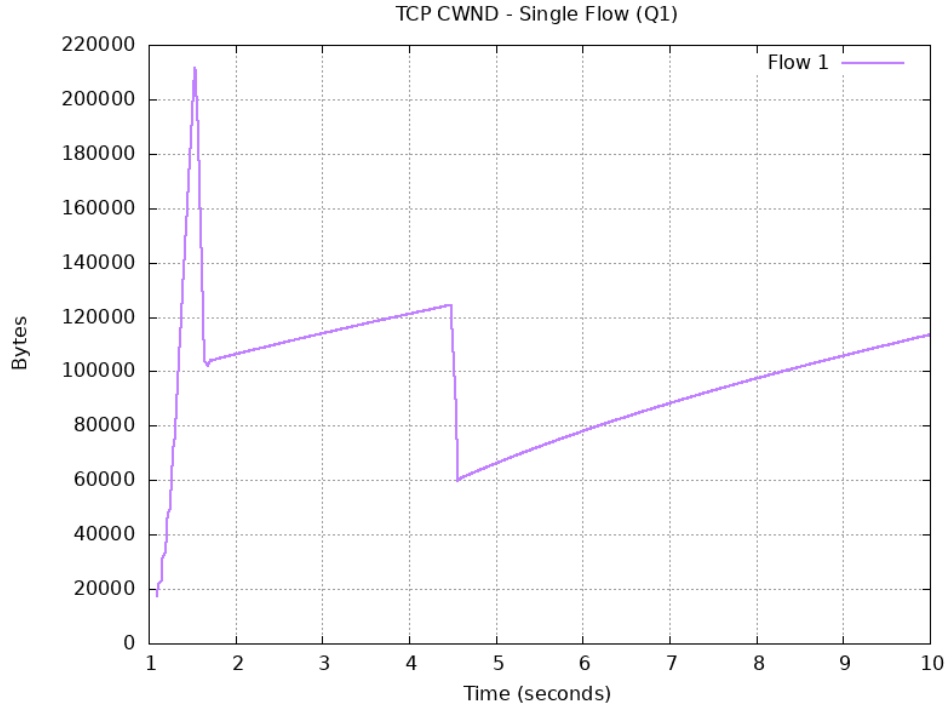


Figure 1: TCP CWND over time for a single flow (Node 0 to Node 2). The sharp drop indicates a packet loss event, followed by linear AIMD recovery.

## Q2. Two Competing Flows

**Setup:** Added a second flow ( $n1 \rightarrow n3$ ). Both share the bottleneck.

### a. Are the flows getting equal bandwidth?

Yes. Observing the CWND behaviors and average throughputs, both flows achieve approximately equal bandwidth (roughly 4.5 to 4.7 Mbps each), evenly splitting the bottleneck capacity.

### b. Why does TCP tend toward fair sharing?

TCP achieves fair sharing due to its **AIMD (Additive Increase, Multiplicative Decrease)** algorithm. Both flows increase their congestion windows at the same linear rate (+1 MSS per RTT) when the network is uncongested. When the bottleneck router's queue overflows and drops packets, both flows detect the loss and cut their windows in half. Because the flow with the larger window loses more of its bandwidth in absolute terms during the multiplicative decrease phase, the two flows gradually converge toward an equal equilibrium.

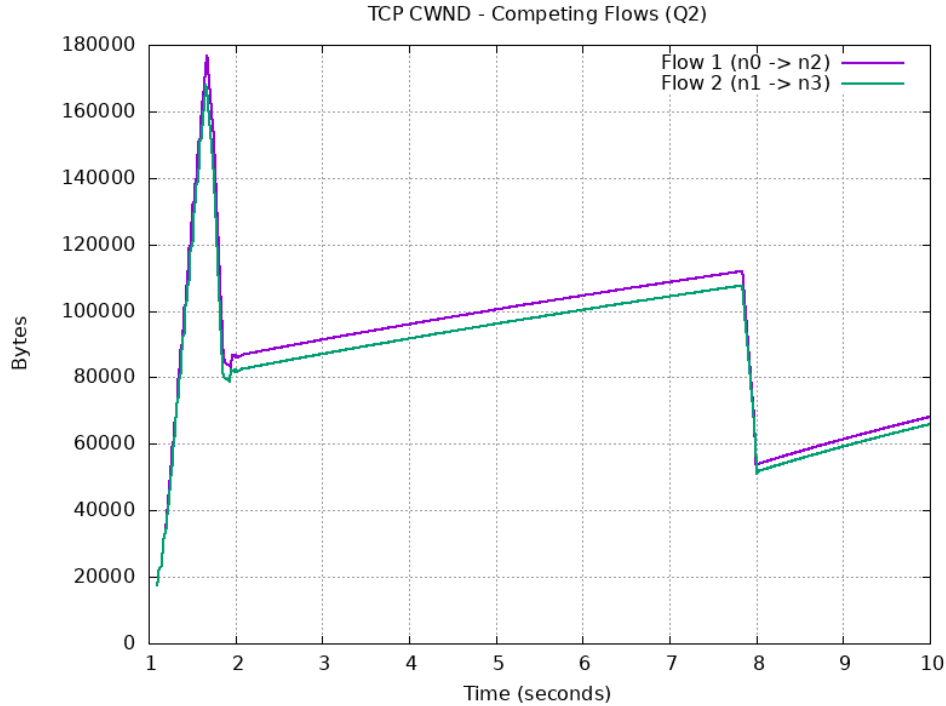


Figure 2: Two competing TCP flows sharing the 10 Mbps bottleneck. Their CWND graphs overlap closely, demonstrating TCP's convergence toward fair sharing.

### Q3. Multiple Parallel Connections

**Setup:** User 1 runs 3 TCP flows from  $n0 \rightarrow n2$ . User 2 runs 1 TCP flow from  $n1 \rightarrow n3$ .

**c. Does User 1 get more bandwidth?**

Yes, User 1 receives significantly more bandwidth. As shown in the simulation results (Figure 3), User 1 achieved a total throughput of **7.33 Mbps**, while User 2 achieved only **2.42 Mbps**.

**d. Does TCP provide per-flow or per-user fairness?**

TCP provides **per-flow fairness**, not per-user fairness. Because the bottleneck link simply sees 4 distinct, competing TCP streams, it applies AIMD evenly across all 4 streams. Each flow receives roughly 1/4 of the bandwidth (approx. 2.4 Mbps). Since User 1 opened 3 distinct flows, they successfully hijacked 3/4 of the total bottleneck bandwidth.

```

=====
=== Simulation Results (3 Flows vs 1 Flow) ===
=====
Flow 1 (10.1.1.1 -> 10.1.4.2) - 2.45424 Mbps
Flow 2 (10.1.1.1 -> 10.1.4.2) - 2.38741 Mbps
Flow 3 (10.1.1.1 -> 10.1.4.2) - 2.49661 Mbps
Flow 4 (10.1.2.1 -> 10.1.5.2) - 2.42772 Mbps

--- Final Bandwidth Breakdown ---
User 1 (Node 0 - 3 Flows) Total: 7.33826 Mbps
User 2 (Node 1 - 1 Flow ) Total: 2.42772 Mbps
=====

```

Figure 3: Terminal output demonstrating the throughput breakdown. Each flow gets approximately 2.4 Mbps, resulting in User 1 consuming 3x more overall bandwidth.

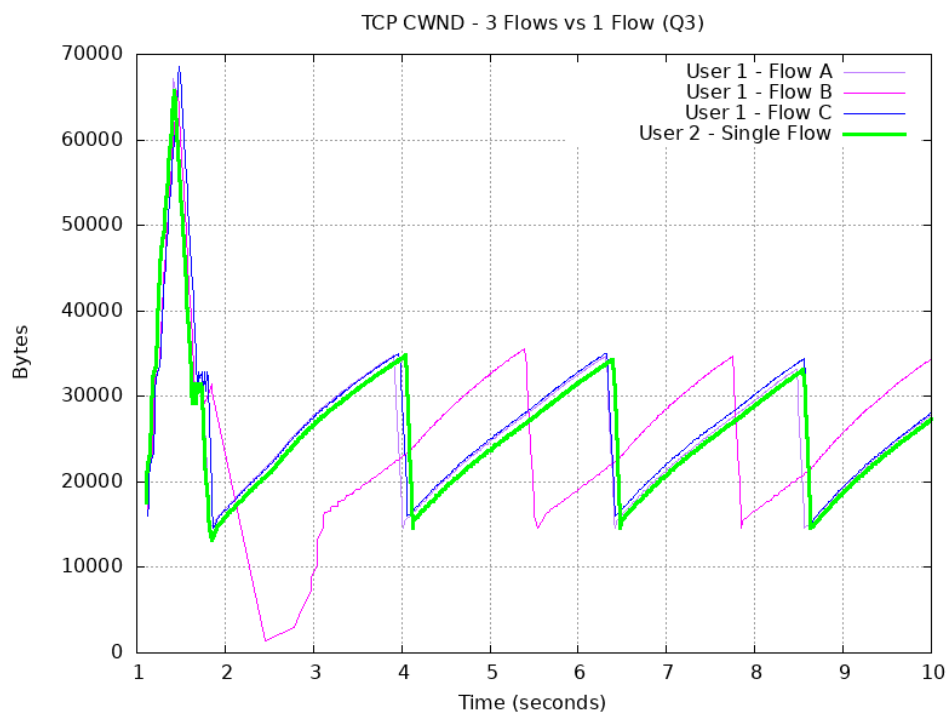


Figure 4: CWND behavior for 3 Flows vs 1 Flow. All four flows follow the same AIMD pattern independently.

## Q4. TCP Variant Comparison (NewReno vs CUBIC)

**Setup:** Simulating a high-latency bottleneck, comparing standard TCP NewReno against TCP CUBIC.

**a. Do variants behave differently? Compare their CWND using a plot.**

Yes, they behave fundamentally differently, as clearly shown in Figure 5. TCP NewReno

(Purple) follows a strict linear growth phase after a drop, taking a significant amount of time to recover its window size on a high-latency link. In contrast, TCP CUBIC (Green) uses a cubic function governed by the time since the last packet drop rather than RTT. It quickly ramps its window back up to near the pre-drop level, flattens out to probe carefully, and then curves upward aggressively if no drops occur.

**b. Which is more aggressive?**

**TCP CUBIC** is much more aggressive. By using a cubic growth curve, it does not wait for slow ACKs over a high-latency link to increase its window. It reclaims available bandwidth much faster than NewReno's linear approach.

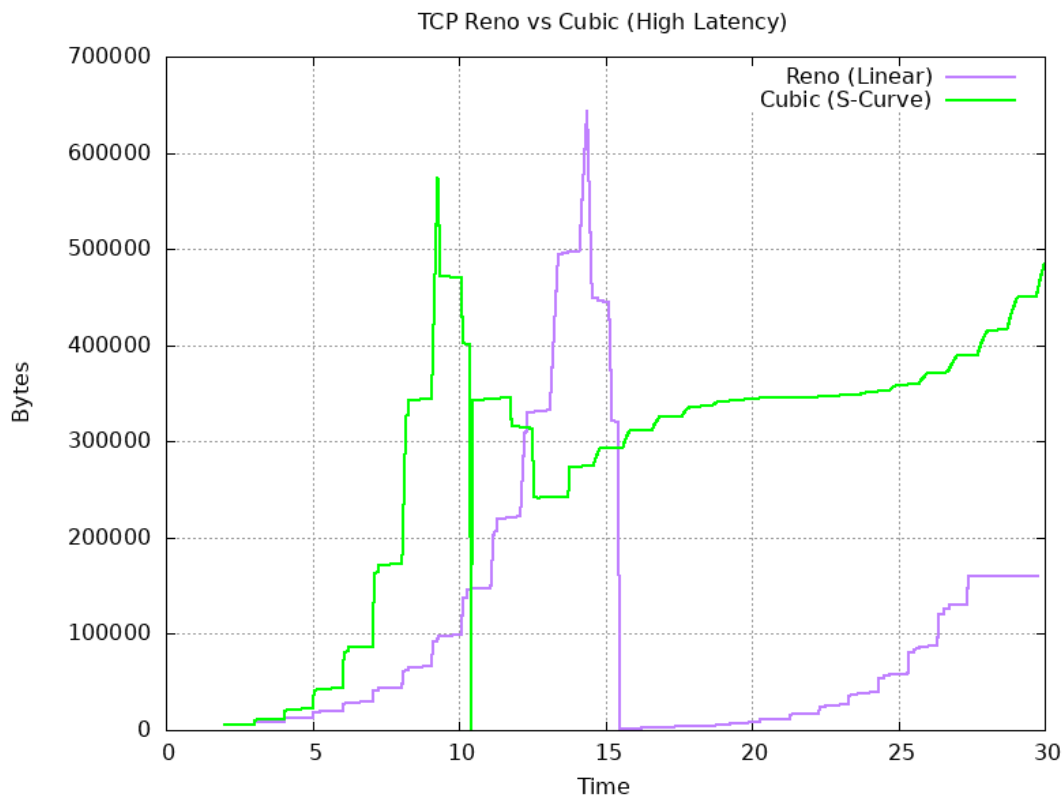


Figure 5: TCP Reno vs Cubic over a High Latency link. Note Reno's slow linear recovery compared to CUBIC's aggressive S-curve shape.